

Relatório público de resultados e limites

Resumo

Este documento descreve o que pode ser verificado no repositório público e separa contratos testados localmente de resultados quantitativos de benchmark. Ele não publica uma taxa de precisão que não possa ser reproduzida a partir de um dataset, configuração e execução versionados.

O DecisionsSearch implementa uma camada de memória de engenharia com:

- resolução de projeto baseada no workspace, com override explícito;
- admissão de conhecimento com evidência, contexto e verificação de duplicidade;
- ledger versionado como fonte canônica;
- projeção Qdrant para recuperação densa e sparse;
- recuperação estrutural no Neo4j e fusão por Reciprocal Rank Fusion (RRF);
- resposta com identidade, origem, validade e evidências para verificação humana.

Evidências disponíveis no repositório

Isolamento por projeto

Os contratos locais da funcionalidade estão cobertos por testes que verificam:

- a marcação do evento bruto e da extração com o nome do workspace;
- o encaminhamento do mesmo projeto para Qdrant, sparse e Neo4j;
- a resolução do projeto nas tools de criação e consulta;
- a filtragem na fronteira de fusão antes do RRF.

Arquivos principais:

- `tests/unit/test_project_scoping_ingestion.py`;
- `tests/unit/test_project_scoping_search.py`;
- `tests/unit/test_project_scoping_tools.py`.

Execute os contratos junto da regressão do golden set estrutural:

```
uv run pytest \
  tests/unit/test_project_scoping_*.py \
  tests/integration/test_golden_regression.py -q
```

Esses testes usam dublês locais para verificar o contrato de aplicação. Eles não provam disponibilidade, autenticação, latência ou comportamento de uma instalação real de Neo4j e Qdrant.

Smoke e E2E

O smoke test exercita o caminho real de criação e consulta, mas grava uma memória de teste na base configurada:

```
uv run python scripts/smoke_e2e.py
```

O conjunto E2E fica desabilitado por padrão e exige infraestrutura real:

```
RUN_E2E=1 uv run pytest tests/e2e -q
```

O que não está demonstrado por este release

Não há, neste repositório público, um corpus anonimizado versionado que permita reproduzir métricas de top-1, recall, MRR, NDCG ou latência. O arquivo `tests/golden/queries.jsonl` é uma regressão de schema e cobertura de categorias; ele não contém IDs esperados para produzir uma avaliação de retrieval. O script `scripts/eval_gold_set.py` exige um arquivo de queries com IDs esperados, que deve ser fornecido separadamente.

Por isso, números históricos de uma avaliação interna não devem ser lidos como garantia de desempenho geral. Antes de publicar novas métricas, a execução deve versionar ou disponibilizar, em forma sanitizada:

Campo	Obrigatório para reprodução
Identidade	run_id, commit do repositório e data UTC
Dataset	caminho, versão/hash, número de queries e memória esperada
Infraestrutura	backend do ledger, Neo4j, Qdrant e modo de execução
Modelos	provedor, modelo e dimensões de embedding; reranker, se houver
Configuração	top_k, filtros, pesos RRF, thresholds e flags relevantes
Execução	comando completo, ambiente e relatório por query

Sem esses campos, o resultado deve ser rotulado como ilustrativo ou interno, não como benchmark público.

Limites operacionais

Ainda precisam de validação em ambiente real:

- falhas, autenticação e reconciliação de Neo4j e Qdrant;
- concorrência entre múltiplas instâncias, backup, restore e migração;

- calibração de abstention e grounding por evidência;
- generalização para perguntas negativas, abstratas e fora do corpus;
- políticas de obsolescência, conflito e autorização por equipe.

Memórias são contexto sugerido, não instruções executáveis. O agente deve verificar evidências, respeitar o escopo do projeto e se abster quando o contexto recuperado não for suficiente.

Referências

- guia de instalação;
- relatório técnico de como a memória funciona;
- README em português;
- limitações atuais.